

2025-1

Basic Algorithm Study → For Beginners

기초 알고리즘 스터디 4주차

01

메모이제이션

· 정의

02

DP

· 탑다운

· 바텀업

03

문제 풀어보기

메모이제이션

01

메모이제이션은 가장 간단한 최적화 기법 중 하나입니다.

중복된 값을 구할 때 처음부터 다시 계산하는 대신
이전에 저장했던 값을 기억해서 재사용하면 됩니다.



01

메모이제이션이 뭐지?

재귀함수를 이용해서 피보나치 수열을 계산해 봅시다.

$$P[i] = P[i-1] + P[i-2]$$



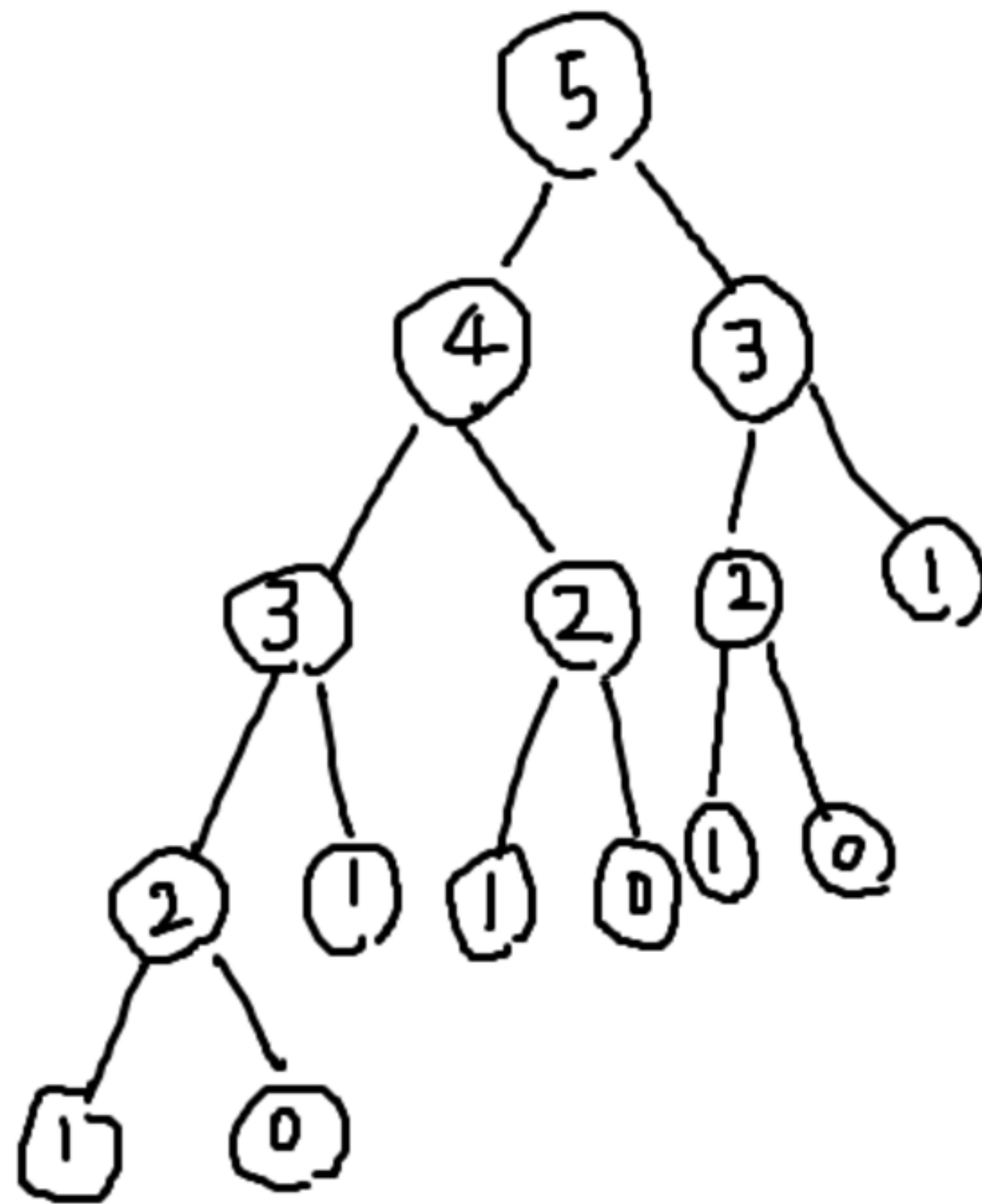
01

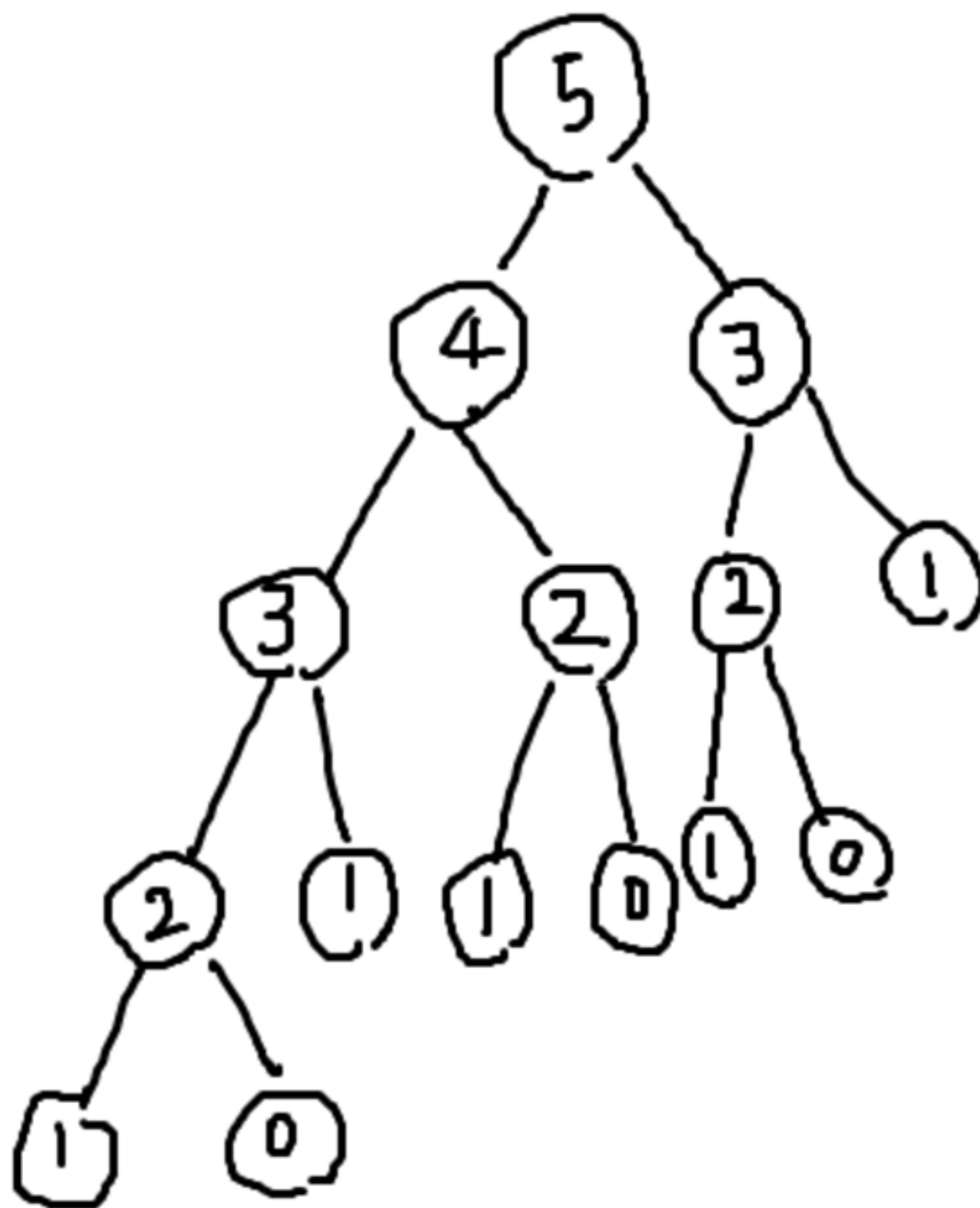
메모이제이션이 뭐지?

```
def fibo(i):  
    if i == 0:  
        return 0  
    elif i == 1:  
        return 1  
    return fibo(i-1)+fibo(i-2)
```

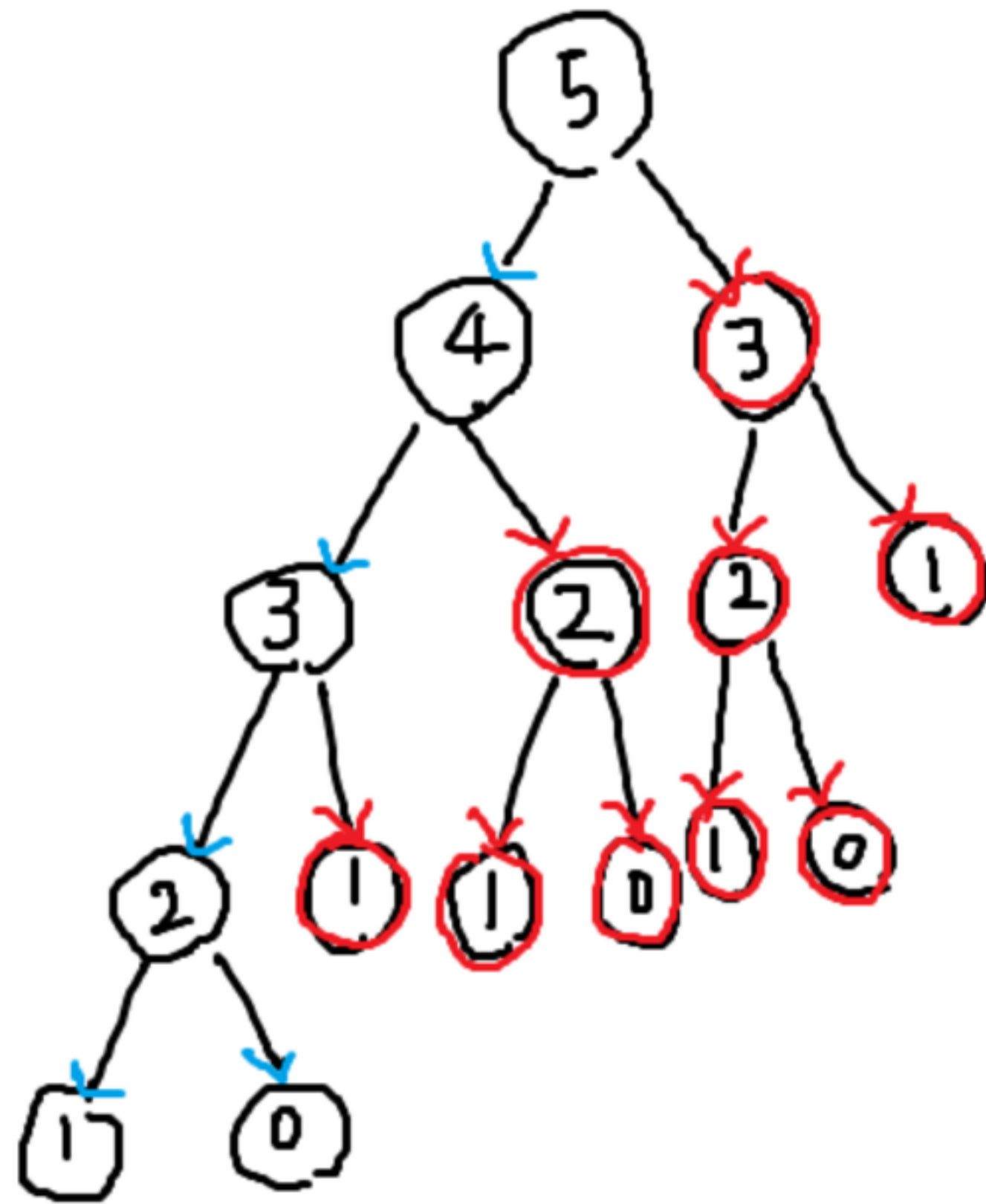

01

메모이제이션이 뭐지?





재귀 깊이: N , 매 깊이마다 2갈래로 나뉘지므로 $O(2^N)$





01

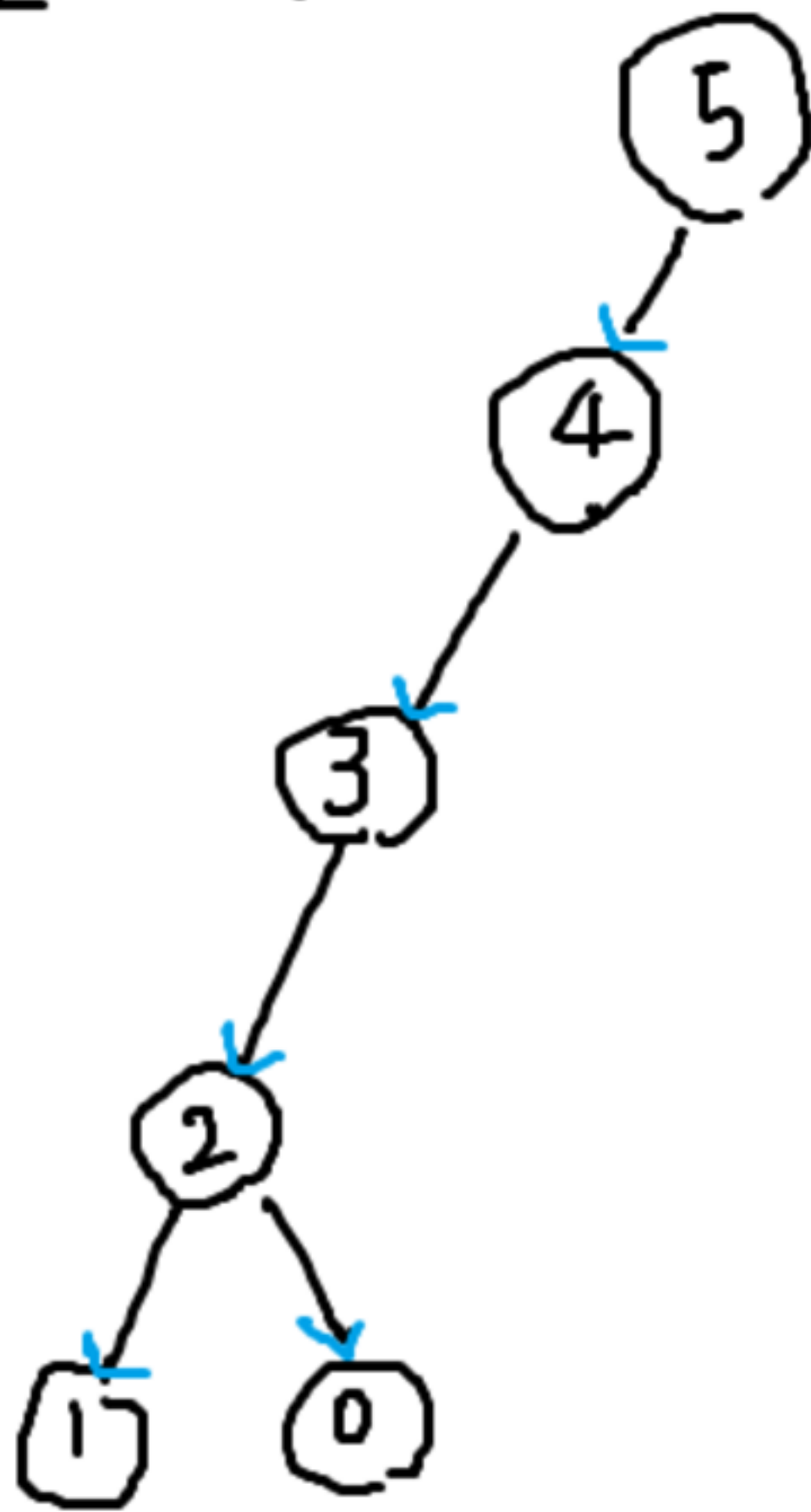
메모이제이션이 뭐지?

앞서 구했던 값을 전부 저장해두고
같은 값이 또 나오면 재귀 대신 저장해둔 값을 재사용하자.

01

메모이제이션이 뭐지?

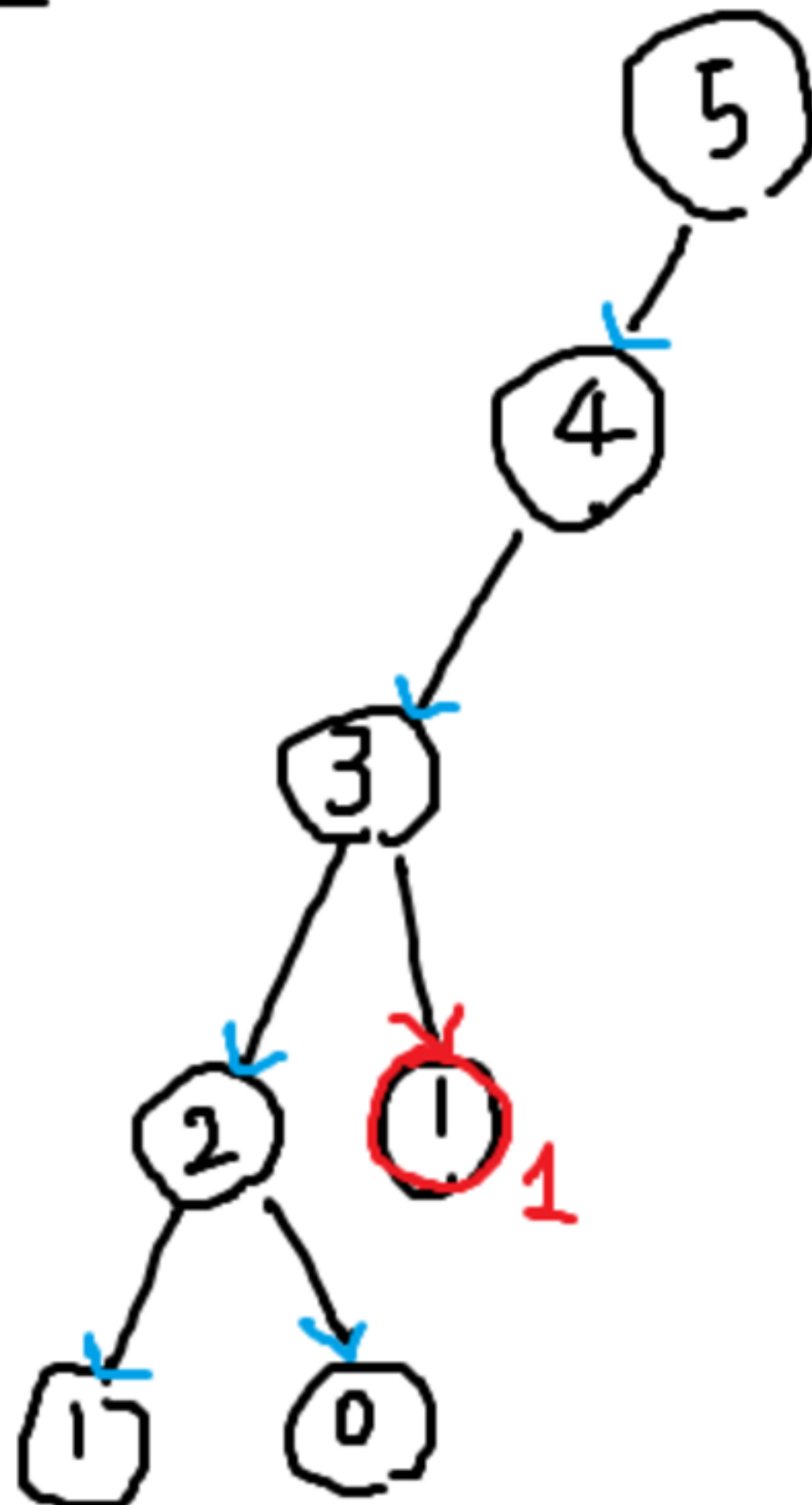
$[0, 1, 1, -1, -1, -1]$



01

메모이제이션이 뭐지?

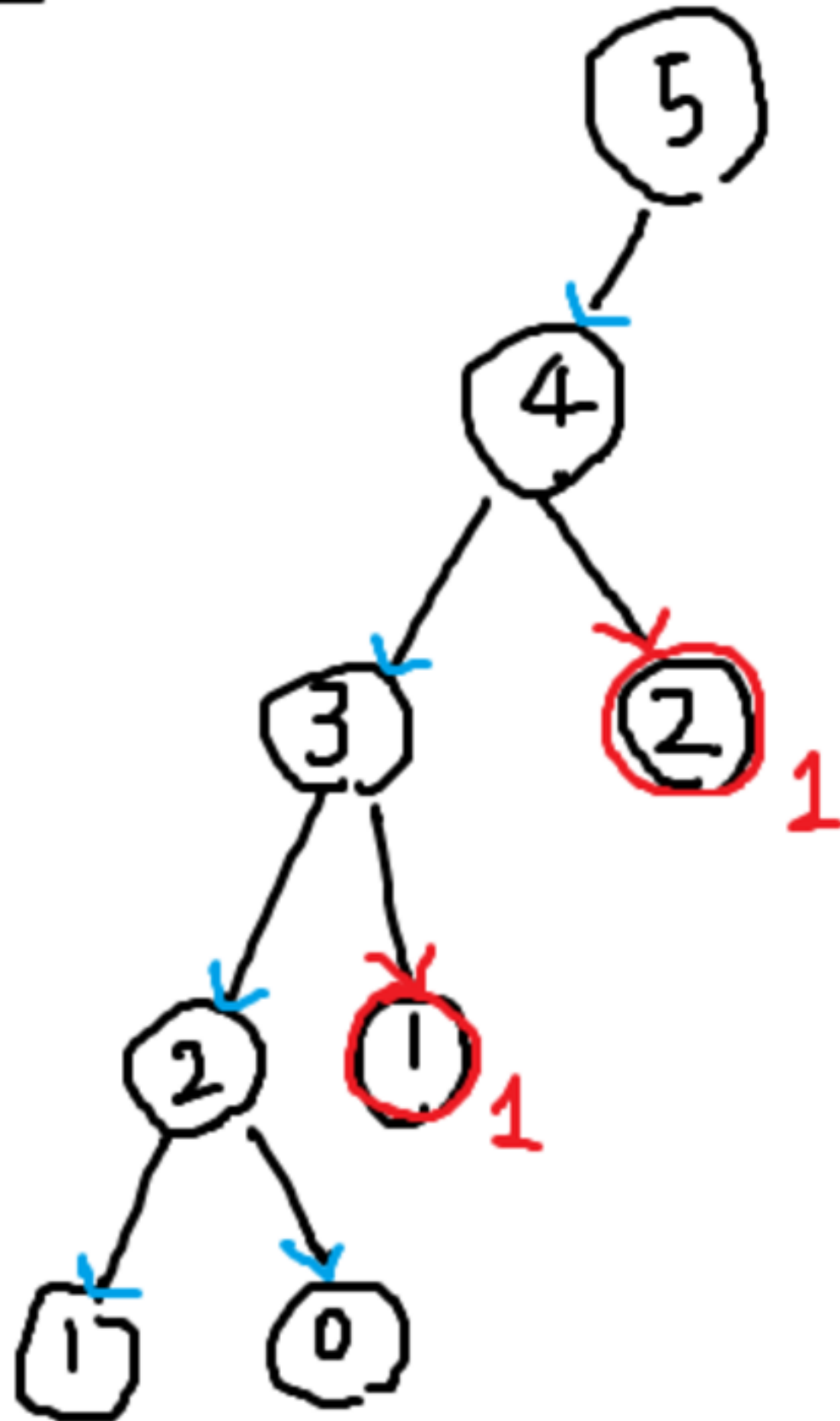
$[0, 1, 1, 2, -1, -1]$



01

메모이제이션이 뭐지?

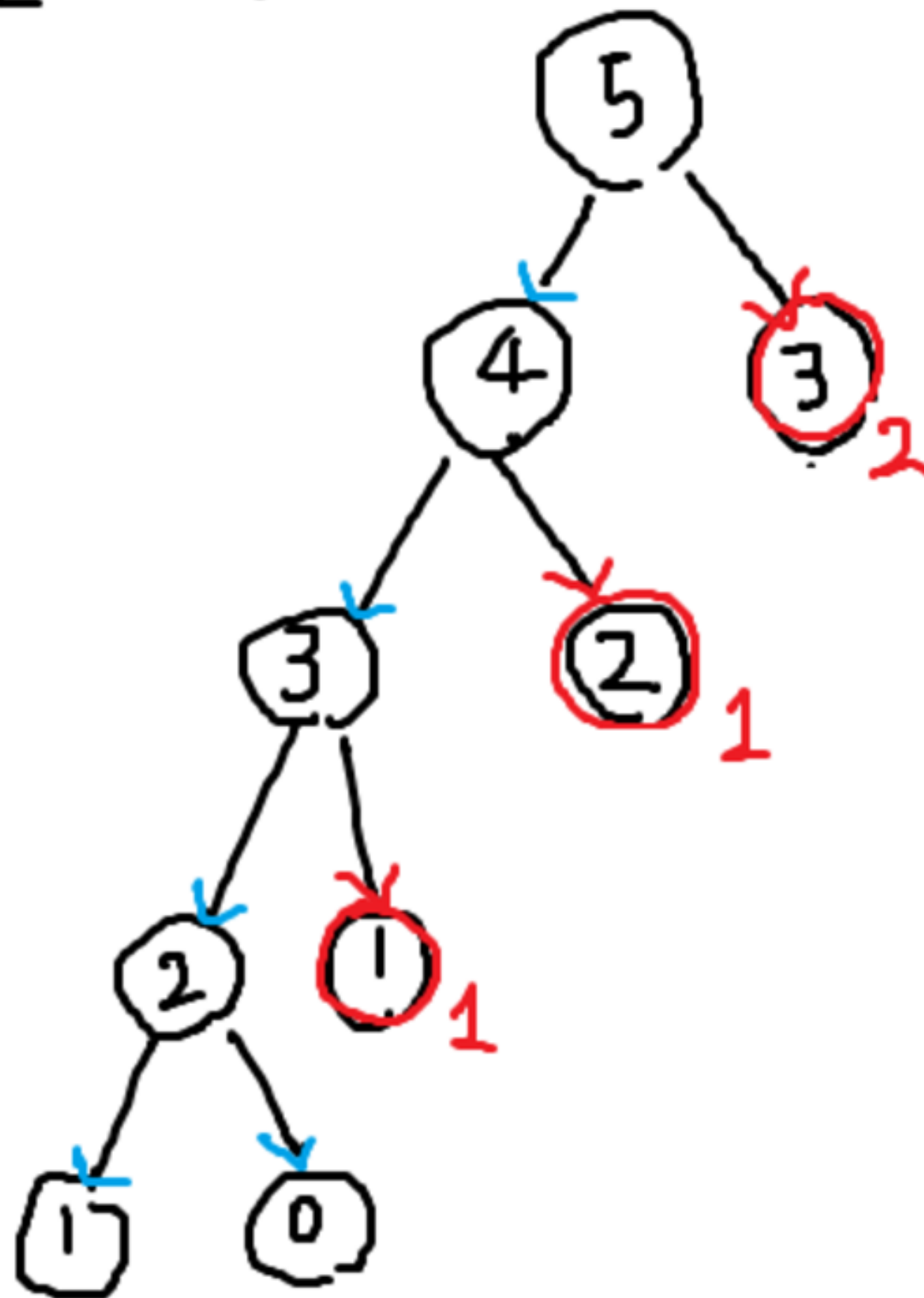
$[0, 1, 1, 2, 3, -1]$



01

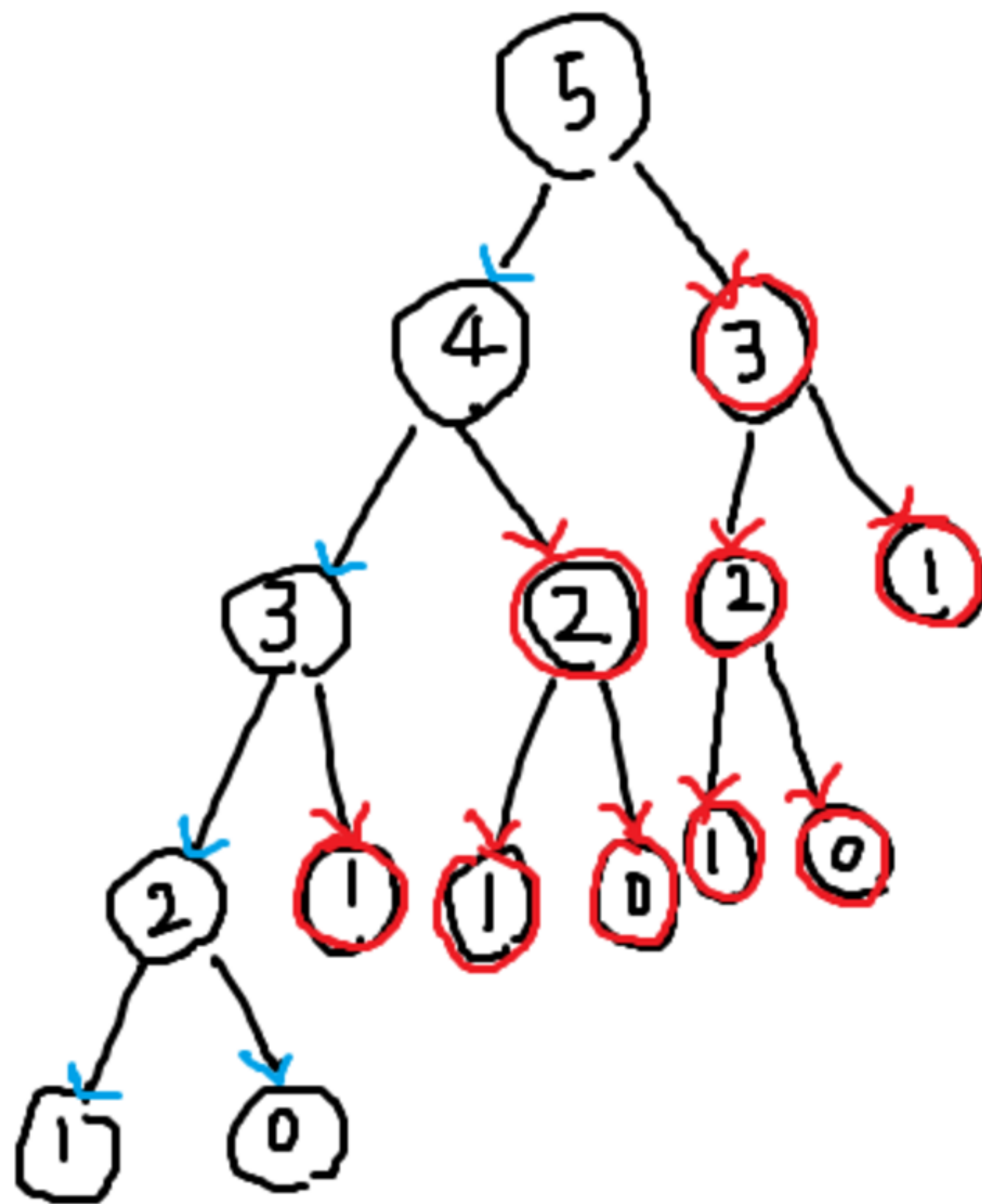
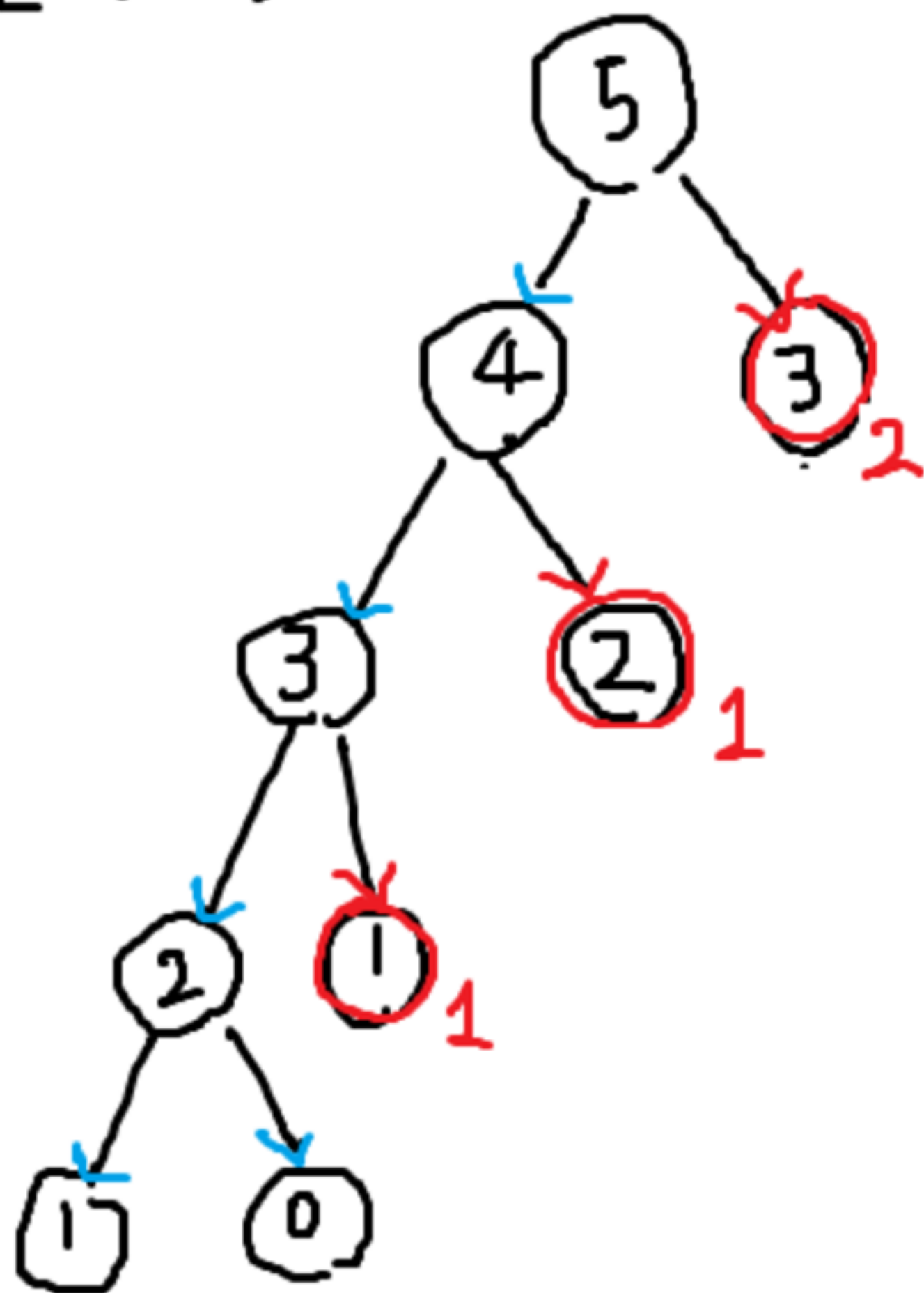
메모이제이션이 뭐지?

[0, 1, 1, 2, 3, 5]



01

메모이제이션이 뭐지?

 $[0, 1, 1, 2, 3, 5]$ 

01

메모이제이션이 뭐지?

```
fib = [-1]*21
fib[0], fib[1] = 0, 1

3 개의 사용 위치
def fibo(i):
    if fib[i] != -1:
        return fib[i]
    return fibo(i-1)+fibo(i-2)
```


01

메모이제이션이 뭐지?

재귀 깊이: N , 매 깊이마다 2갈래로 나뉘지므로 $O(2^N)$



재귀 깊이: N , 깊이 1당 노드가 2개이므로 $O(N)$



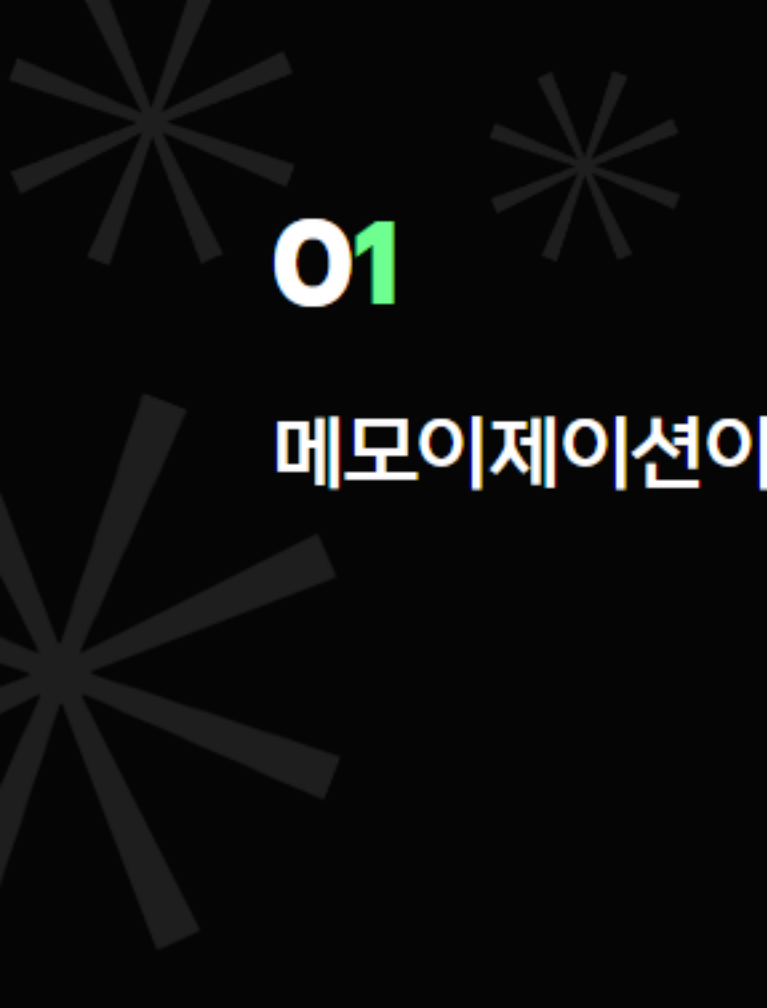
01

메모이제이션이 뭐지?

$N = 20$ 일 때

$O(N) = 20, O(2^N) = 1,048,576$

N 이 커질수록 효율 차이는 점점 커진다.



01

메모이제이션이 뭐지?

같은 정보를 중복해서 사용할수록
메모이제이션을 사용하는 것이 유리하다.

배열에서 값을 꺼내는 연산은 $O(1)$

피보나치 수를 다시 계산하는 것보다 훨씬 효율적이다!



01

메모이제이션이 뭐지?

메모이제이션을 다양한 알고리즘에 접목하여
효율성을 끌어올릴 수 있다.
주로 백트래킹, DP 등에 사용된다.

DP

—

102



02

DP

DP(Dynamic Programming)

동적 프로그래밍



02

DP

복잡한 문제를 작은 문제로 나눠서
조금씩 풀어나갑시다!

부분 문제의 값을 미리 구해놓고,
그 값을 이용해서 다음으로 큰 문제의 값을 구하고.....
그 값으로 다음 값을 또 구하고....(반복)

이때 부분 문제와 큰 문제를 이어주는 관계식이 바로 **점화식**입니다.
눈덩이를 굴리듯이 **점화식**을 반복하면서 찾고싶은 답이 될때까지
문제를 키워나가면 됩니다.

피보나치 수열을 DP로 구해봅시다.

0	1	-1	-1	-1	-1
---	---	----	----	----	----

$f[0]$, $f[1]$ 의 초기값을 0, 1로 설정합니다.

0	1	-1	-1	-1	-1
---	---	----	----	----	----

피보나치 수열의 점화식

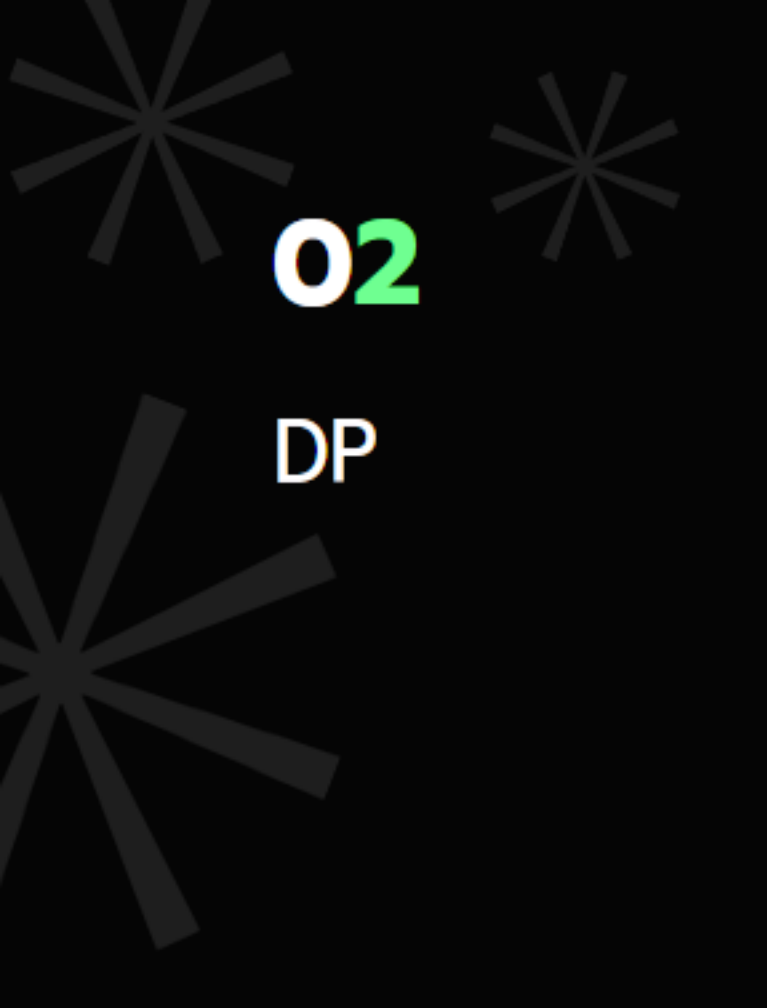
$f[i] = f[i-1] + f[i-2]$ 을 이용해서
값을 키워나가 봅시다.

02

DP

0	1	1	-1	-1	-1
---	---	---	----	----	----

$f[0]$, $f[1]$ 을 이용해서 $f[2]$ 를 $O(1)$ 만에 계산합니다.

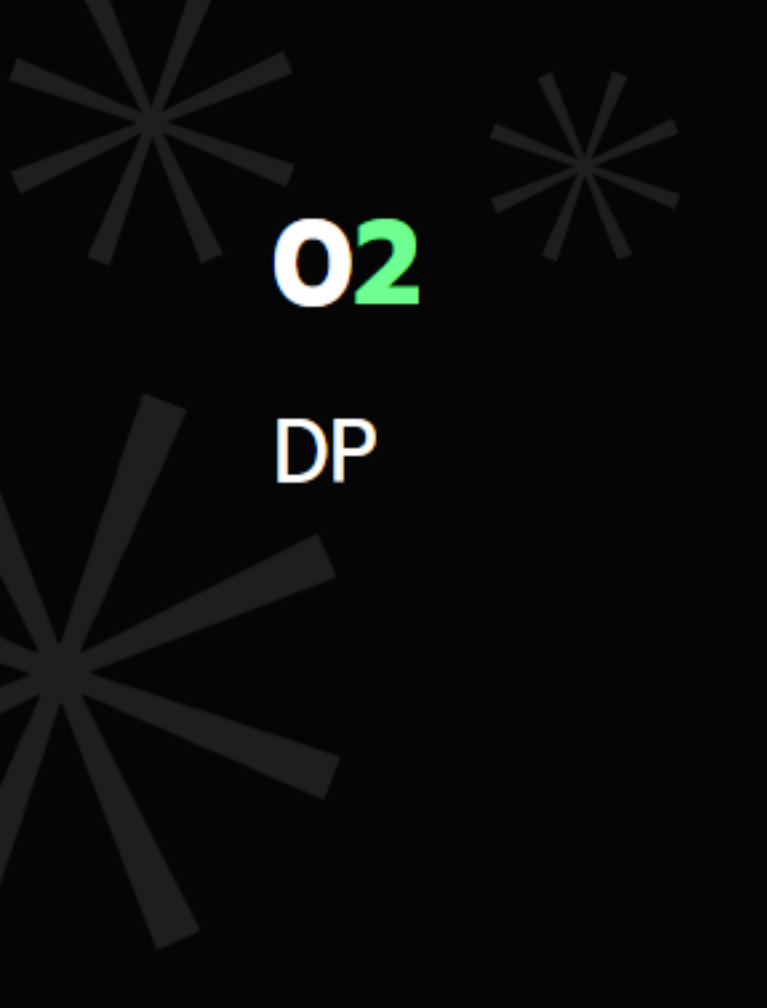


02

DP

0	1	1	2	-1	-1
---	---	---	---	----	----

f[1]과 이전에 만들어둔 f[2]로 f[3]을 $O(1)$ 에 계산합니다.



02

DP

0	1	1	2	3	-1
---	---	---	---	---	----

만들어둔 $f[2]$ 와 $f[3]$ 으로 $f[4]$ 를 $O(1)$ 에 계산합니다.

02

DP

0	1	1	2	3	5
---	---	---	---	---	---

같은 방법으로 $f[5]$ 까지 계산하면 끝!



02

DP

0	1	1	2	3	5
---	---	---	---	---	---

같은 방법을 $O(N)$ 번 반복하면 `fibo[N]`의 값을 구할 수 있습니다.

0	1	1	2	3	5
---	---	---	---	---	---

이런 식으로 값을 찾는 방법을 Bottom-Up DP라고 하고,
처음 만들었던 것처럼 재귀함수로 구현하면 Top-Down DP라고 합니다.

Top-Down

가장 큰 문제에서부터 작은 문제로 쪼개나가기

```
fib = [-1]*21
fib[0], fib[1] = 0, 1
```

3 개의 사용 위치

```
def fibo(i):
    if fib[i] != -1:
        return fib[i]
    return fibo(i-1)+fibo(i-2)
```

Bottom-Up

가장 작은 문제부터 차례차례 반복문으로 쌓아가기

```
fib = [-1]*21
fib[0], fib[1] = 0, 1

for i in range(2, 21):
    fib[i] = fib[i-1] + fib[i-2]

print(fib[6])
```

Top-Down

가장 큰 문제에서부터 작은 문제로 쪼개나가기

문제를 푸는 데 필요한 값만 계산할 수 있다.

재귀함수 기반이라 연산이 느리다.

$f[5] \rightarrow f[4] \rightarrow f[3] \rightarrow f[2] \rightarrow f[1] \rightarrow f[0]$

Bottom-Up

가장 작은 문제부터 차례차례 반복문으로 쌓아가기

범위 내의 모든 값을 계산한다.

반복문 기반이라 재귀함수에 비해 빠르다.

값을 계산하는 방식이 조금 더 직관적이다.

둘 중 문제에 따라 더 편한 방식으로 구현하면 됩니다.
그러나 일반적으로는 바텀 업 방식을 사용하는 경우가 많습니다.

재귀함수로 DP를 구현하면
구하려는 문제가 커질수록 재귀 깊이가 늘어나고
반복문에 비해 메모리를 더 많이 사용합니다.
저도 기본적으로 DP 문제를 풀때는 바텀업을 사용합니다.

문제풀어보기

03

DP의 개념은 이게 끝입니다.

개념이 어렵다기 보다는 문제의 유형이 점화식의 수만큼 많습니다.

dp 문제를 풀때 가장 어려운 부분은 점화식을 정의하는 것입니다.
그 외의 구현은 정말 간단하기 때문에 대부분의 문제는
"dp 문제임을 파악하고 점화식을 찾는 것" 이 포인트입니다.

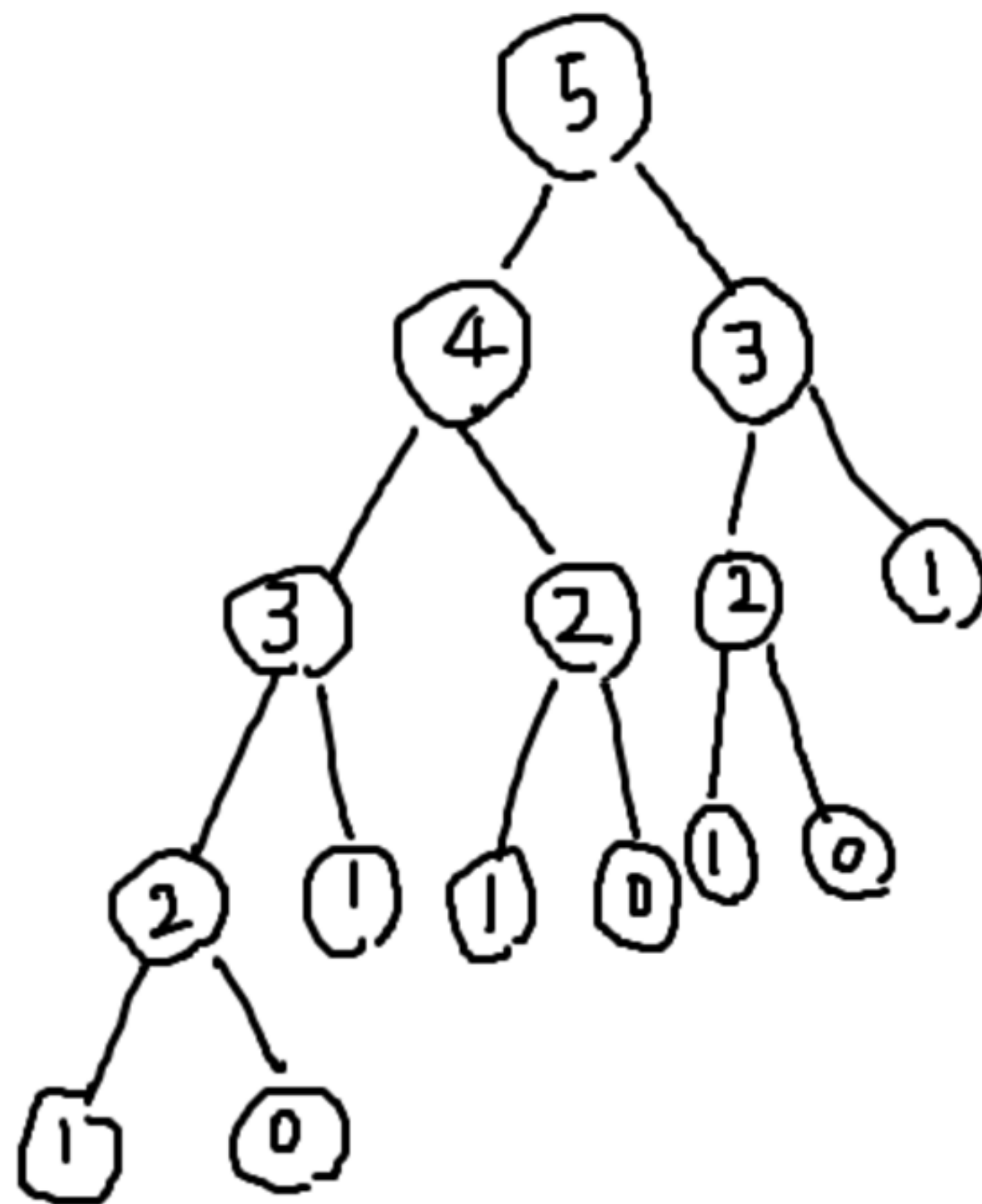


03

문제 풀어보기

예제: 피보나치 함수(#1003)

피보나치 수를 구하라는 문제는 아니고,
무식한 방법으로 피보나치 수를 구할 때
0의 호출 횟수와 1의 호출 횟수를 구하는 문제



$\text{fibonacci}(N)$ 을 호출하면
 $\text{fibonacci}(N-1)$, $\text{fibonacci}(N-2)$ 이 호출됨.

$\text{fibonacci}(N)$ 을 호출하면

$\text{fibonacci}(N-1)$, $\text{fibonacci}(N-2)$ 을 호출한 것과 같은 효과.

메모이제이션을 사용하지 않으므로 각 함수의 0, 1 호출 횟수는 독립적이다.

그렇다면 $\text{fibonacci}(N)$ 을 구할 때 $\text{fibonacci}(0)$, $\text{fibonacci}(1)$ 총 호출 횟수는 $\text{fibonacci}(N-1)$, $\text{fibonacci}(N-2)$ 의 0, 1 호출 횟수를 더한 값과 같을 것이다.

여기서 DP 배열을 만들기 위한 상태를 정의해 보자.
 $DP[i][0]$ = fibo(i)를 호출했을 때 0을 호출한 횟수
 $DP[i][1]$ = fibo(i)를 호출했을 때 1을 호출한 횟수

이게 문제를 푸는 데에 필요한 정보의 전부이다.



03

문제 풀어보기

점화식에 필요한 상태를 정의했으니 점화식을 세워보자.

```
dp = [[0, 0] for _ in range(N+1)]  
dp[0] = [1, 0] / dp[1] = [0, 1]
```

```
dp[i][0] = dp[i-1][0] + dp[i-2][0] #fibonacci(i) 호출 시 fibonacci(0)호출 수  
dp[i][1] = dp[i-1][1] + dp[i-2][1] #fibonacci(i) 호출 시 fibonacci(1)호출 수
```



```
import sys
input = sys.stdin.readline

T = int(input().strip())
dp = [[0, 0] for i in range(41)]
dp[0] = [1, 0]
dp[1] = [0, 1]
for i in range(2, 41):
    dp[i][0] = dp[i-1][0] + dp[i-2][0]
    dp[i][1] = dp[i-1][1] + dp[i-2][1]

for _ in range(T):
    print(*dp[int(input().strip())])
```

각 상태의 값은 $O(1)$ 로 처리할 수 있으므로
전체 실행 시간은 상태의 수에 비례할 것이다.

이 문제의 시간복잡도는 $O(N)$
더 자세히 표현하자면 $O(2N)$



03

문제 풀어보기

예제: 01타일 (#1904)



03

문제 풀어보기

다짜고짜 길이 N 의 수열을 채우는 방법을 생각해 보자.

00타일이 0개일 때

00타일이 1개일 때

00타일이 2개일 때

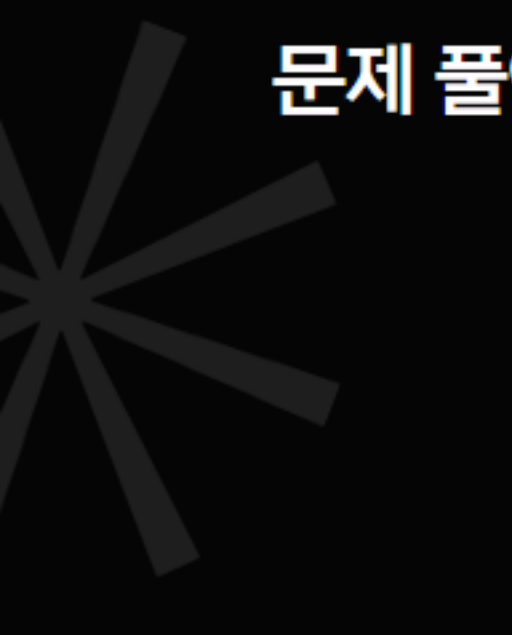
...

모든 경우를 나눠서 조합을 계산할까?



03

문제 풀어보기



%15746도 계산해야 해서 쉽지 않을 것 같다.
다른 좋은 방법을 찾아보자!

길이 1의 타일을 채우는 경우의 수: 1, [1]
길이 2의 타일을 채우는 경우의 수: 2, [11, 00]

길이 3의 타일을 채우는 경우의 수: ?

길이 1의 타일을 채우는 경우의 수: 1, [1]
길이 2의 타일을 채우는 경우의 수: 2, [11, 00]

길이 3의 타일을 채우는 경우의 수:
길이 1의 타일에 '00'을 붙이거나,
길이 2의 타일에 '1'을 붙이거나

길이 N의 타일을 채우는 방법은

길이 N-1의 타일 + '1'

또는 길이 N-2의 타일 + '00'

$$dp[i] = dp[i-1] + dp[i-2]$$

포장만 열심히 해봤을 뿐 피보나치 수열과 똑같다.



03

문제 풀어보기

어떻게 이럴 수 있을까?

- 길이 N 의 수열을
1. 마지막 수가 1인 경우
 2. 마지막 수가 0인 경우
- 둘로 나눠보자.

마지막 수에 1을 먼저 채웠을 때
나머지 $N-1$ 길이의 수열을 채우는 방법의 수는
 $dp[N-1]$ 이다.

마찬가지로 마지막 두 수를 00으로 채웠을 때는
N-2 길이의 수열을 마저 채우면 된다.
이 값은 $dp[N-2]$ 와 같다.

타일을 붙이는 경우의 수를 구할 때
그 전까지 어떤 모양으로 타일을 붙였는지는 중요치 않다.
중요하지 않은 정보는 걸러내고, 필요한 정보만 메모이제이션한다.

여기서 잠깐.

이 문제에서 최대 수열의 길이는 1,000,000
이만큼 피보나치 수열이 길어지면 수가 너무 커져서
계산하는데 시간이 매우매우매우 오래 걸린다.

파이썬이 아닌 다른 언어일 경우 오버플로우가 발생해서 연산이 불가능하다.

15746으로 나눈 나머지라는 부분에 주목하자.
나머지(%)연산은 다음과 같은 법칙이 성립한다.

$$(a + b) \% m = ((a \% m) + (b \% m)) \% m$$

$$(a - b) \% m = ((a \% m) - (b \% m)) \% m$$

$$(a * b) \% m = ((a \% m) * (b \% m)) \% m$$

m 으로 나눈 나머지를 구할 때는
덧셈을 할 때마다 $\%m$ 을 미리 해도 같은 결과를 얻을 수 있다.

이를 기반으로 점화식을 바꿔보자.



03

문제 풀어보기

$$dp[i] = (dp[i-1] + dp[i-2]) \% 15746$$

```
import sys

sys.setrecursionlimit(10**6)
input = sys.stdin.readline

N = int(input())
lst = [0]*(N+3)
lst[1], lst[2] = 1, 2
if N == 1 or N == 2:
    print(lst[N])
else:
    for i in range(3, N+1):
        lst[i] = (lst[i-1] + lst[i-2]) % 15746
    print(lst[N])
```




03

문제 풀어보기

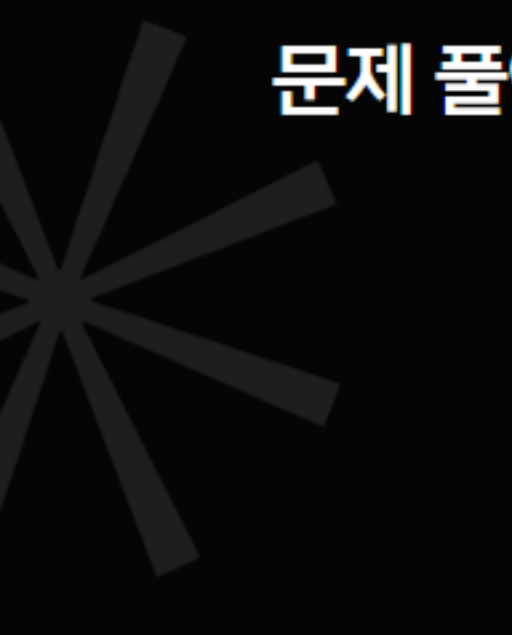
예제: 돌 게임 (#9655)

배스킨라빈스 31 게임과 비슷해 보이는데,
먼저 관찰부터 해 봅시다.



03

문제 풀어보기



돌이 1, 3개일 때: 선공 승리

돌이 2개일 때: 후공 승리

턴이 돌아왔을 때 할 수 있는 행동은
돌을 1개 집거나, 3개 집거나 둘 중 하나

돌 N개가 있을 때

1개를 집었을 때: 돌이 $N-1$ 인 상태에서 상대 턴 시작

3개를 집었을 때: 돌이 $N-3$ 인 상태에서 상대 턴 시작

돌이 $N-1$, $N-3$ 일 때 상대 턴으로 넘어갔으므로
상대 입장에서는 돌이 $N-1$, $N-3$ 개였을 때 선공을 잡은 상황과 똑같다.

$dp[i]$: 초기에 돌이 i 개였을 때 선공이 이기면 1, 지면 0
플레이어 입장에서는 항상 최선의 선택을 하므로
 $N-1, N-3$ 중에서 턴을 넘긴다면 선공이 지는 상태,
즉 상대가 선공일 때 내가 이길 수 있는 방법을 선택할 것이다.



03

문제 풀어보기

$dp[i] = dp[i-1], dp[i-3]$ 중 한 개라도 0이면 1 아니면 0



03

문제 풀어보기

$dp[1] = 1$

$dp[2] = 0$

$dp[3] = 1$



03

문제 풀어보기

$$dp[1] = 1$$

$$dp[2] = 0$$

$$dp[3] = 1$$

$$dp[4] = dp[3], dp[1] \text{이 전부 } 1 \text{이므로 } 0$$



03

문제 풀어보기

$dp[1] = 1$

$dp[2] = 0$

$dp[3] = 1$

$dp[4] = 0$

$dp[5] = 1$



03

문제 풀어보기

$dp[1] = 1$

$dp[2] = 0$

$dp[3] = 1$

$dp[4] = 0$

$dp[5] = 1$

$dp[6] = 0$

```
def solve():  
    N = int(input().strip())  
    dp = [0]*(N+1)  
    dp[1], dp[3] = 1, 1  
    for i in range(4, N):  
        if min(dp[i-1], dp[i-3]) == 0:  
            dp[i] = 1  
        else:  
            dp[i] = 0  
    print("SK" if dp[N] == 1 else "CY")
```


$$dp[1] = 1$$

$$dp[2] = 0$$

$$dp[3] = 1$$

$$dp[4] = 0$$

$$dp[5] = 1$$

$$dp[6] = 0$$

가만 보니 그냥 N 이 짝수면 0, 홀수면 1이다.

```
import sys
input = sys.stdin.readline

N = int(input().strip())

print('SK' if N%2 == 1 else 'CY')
```

더 알아볼만한 내용

이런 류의 게임을 일반화한 정리가 있습니다
스프라그-그런디 정리라고..

주의: 저도 다 아는 건 아님

어우 진짜 너무 궁금해서 미치겠다 싶은 다이아 꿈나무 분들만 찾아보세요
몰라도 사는 데에 전혀 지장이 없습니다.



03

문제 풀어보기

예제: 평범한 배낭(#12685)

(아마 여기부터 강의시간 내에 설명하진 못할 것 같으니 참고자료 느낌으로 봐주시면 되겠습니다)

브루트포스로 해결해 볼까요?
물품의 수가 최대 100개이므로
각각의 물건을 넣는다/넣지 않는다고 나눠서 모든 경우의 수를 탐색하려면
 2^{100} 번 연산을 수행해야 하는데...
말도 안 되겠죠?

그렇다면 어떤 좋은 방법이 있느냐...

dp를 활용해 봅시다.

여기서 필요한 정보는 가치, 무게
가방에 정확히 어떤 물건이 들어갔는지는 굳이 저장하지 않아도 됩니다.

이 문제는 유명한 dp 유형 중 하나로 흔히들 '배낭 문제'라고 합니다.

우선 $dp[i]$ 를 i 번째 물건까지 확인했을 때로 정의하려고 하는데...

물건에 딸린 정보가 **두 개**나 있습니다.

당연히 **무게는 적을수록, 가치는 많이 나갈수록** 좋은 케이스겠죠?

어떤 두 케이스 a, b 에서 무게는 같지만 b 의 가치가 더 크다면 a 케이스는 절대 답이 될 수 없으므로 저장할 필요가 없습니다. 그렇다면...

$dp[i]$: i 번째 물건까지 확인했을 때 **무게의 최솟값**

$dp[i]$: i 번째 물건까지 확인했을 때 **가치의 최댓값**

둘 중 무엇으로 정의해야 할까요?

둘 중 하나만 고를 필요가 없습니다!
우리는 2차원 리스트를 정의하는 방법을 배웠어요
그러면 이 둘 중에 비교해 봅시다.

$dp[i][j]$: i 번째 물건까지 확인했고 무게가 j 일 때 가치의 최댓값

$dp[i][j]$: i 번째 물건까지 확인했고 가치가 j 일 때 무게의 최솟값

$dp[i][j]$: i 번째 물건까지 확인했고 무게가 j 일 때 가치의 최댓값

$dp[i][j]$: i 번째 물건까지 확인했고 가치가 j 일 때 무게의 최솟값

만약 전자를 택했다면 물건 100개를 확인했을 때 무게의 범위는

$\sim 10^7$ (무게의 최댓값 $10^5 * \text{물건의 최대 수 } 10^2$)

이 크기의 배열을 만들기 위해서는 $N*N*W$

즉 10^9 크기의 배열이 필요한데,

메모리 제한 $512\text{MB} = 536,870,912\text{B}$

게다가 10^9 크기의 배열을 돌기 위해서는 10^9 만큼의 시간이 필요하다...

$dp[i][j]$: i 번째 물건까지 확인했고 무게가 j 일 때 가치의 최댓값

$dp[i][j]$: i 번째 물건까지 확인했고 가치가 j 일 때 무게의 최솟값

하지만 준서가 버틸 수 있는 최대 무게를 기준으로 삼는다면
계산 결과 무게를 벗어났을 때 배열에 접근하지 않고 값을 버려버리면 된다.

이 경우 10^5 (버틸 수 있는 최대 무게) * 100개의 물건
크기의 배열을 만들면 된다.

$dp[i][j]$: i 번째 물건까지 확인했고 무게가 j 일 때 가치의 최댓값

$dp[i][j]$: i 번째 물건까지 확인했고 가치가 j 일 때 무게의 최솟값

후자를 택한다면 물건 100개를 확인할 때 가치의 범위는

$$1000 * 100 = 10^5$$

100개의 물건을 확인하면 되므로 $10^5 * 100 = 10^7$

이 방법도 가능해 보인다.

그럼 점화식은?

$dp[i][j]$: i 번째 물건까지 확인했고 무게가 j 일 때 가치의 최댓값
일 경우

$$dp[i][j] = \max(dp[i-1][j], dp[i-1][j - (i\text{번째 물건의 무게})] + (i\text{번째 물건의 가치}))$$

$dp[i][j]$: i 번째 물건까지 확인했고 가치가 j 일 때 무게의 최솟값
일 경우

$$dp[i][j] = \min(dp[i-1][j], dp[i-1][j - (i\text{번째 물건의 가치})] + (i\text{번째 물건의 무게}))$$

여기서 주의할 점은

1. 빨간색 부분 인덱스가 0 이상일 때만 확인해야 한다는 것
 2. 불가능한 경우가 답에 영향을 미치지 않게 하는 것
- 2번 목표는 dp 배열의 초기값을 조정해서 달성할 수 있다.

둘 중 아무 방식으로나 구현하면 된다.

무게를 인덱스로 구분했을 경우 구하려는 값은
N번째 물건까지 확인했을 때 가방에 들어가는 무게 중 가치의 최대 = $\max(dp[N][:K])$

가치를 인덱스로 구분했을 경우
가방에 들어가는 최대 가치 = $dp[N]$ 내에서 무게가 K 이하인 가장 큰 인덱스

당연하지만 max함수를 쓰는게 더 쉽다.
후자는 심심한 사람만 해보자.

```
INF = 10**9+7
```

1개의 사용 위치

```
def solve():
```

```
    N, K = map(int, input().split())
```

```
    lst = [(0, 0)]
```

```
    for _ in range(N):
```

```
        | lst.append(tuple(map(int, input().split())))
```

```
    dp = [[-INF]*(K+1) for _ in range(N+1)] # 담은 무게 기준
```

```
    dp[0][0] = 0
```

```
    for i in range(1, N+1):
```

```
        | for j in range(K+1):
```

```
            | dp[i][j] = dp[i-1][j]
```

```
            | if j >= lst[i][0]:
```

```
                | dp[i][j] = max(dp[i][j], dp[i-1][j-lst[i][0]] + lst[i][1])
```

```
    print(max(dp[N]))
```

놀라운 사실: 이 문제는 1차원 배열만으로도 풀 수 있다.

각자 고민해보자.

진짜 모르겠다면 질문은 디스코드로...



강의 설문조사

QR 제공 예정

Thank you For Listening

4주차 스터디를 들어주셔서 감사합니다.
뒷풀이에 참여하실 분들만 강의실에 남아주세요!

—

